Industrial Robotics Project Assignment

EXECUTING A TRAJECTORY WITH THE FRANKA ROBOT IN GAZEBO

Giovanni Geraci - Matricola 146376

1 INTRODUCTION

The projects consist in making the Franka Emika Robot execute a Lissajous curve (Figure 1) in the Gazebo simulation environment. The geometric equations have been used to create the trajectory, which has been executed in ROS (Robot Operating System) after choosing a suitable controller. During the execution, data on the end-effector are collected to be used later for comparing the desired trajectory with the executed one.

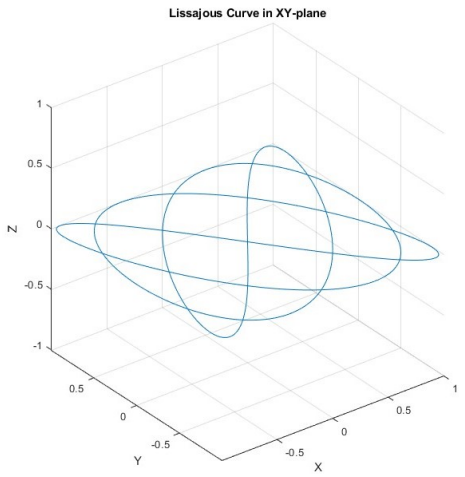


Figure 1: plot of the assigned trajectory.

2 TRAJECTORY DESIGN

The *trajectory design* takes place in the task space and it consists in creating a timed sequence of poses (Cartesian positions and orientations) that the end-effector is commanded to have. This process is done with `trajectory_designer.py`, which creates a series of *waypoints* and interpolates them into a sequence of timed values of poses and saves it in `trajectory.csv`. An initial pose `start_pose` is given and the orientation, expressed with a unit quaternion

which represents the rotation of the end-effector with respect to the base link, is supposed to be fixed.

2.1 Path primitives and generation

The *Lissajous curve* is a curve given by the parametric equations

$$x(t) = A_x \sin(\omega_x t + \phi_x), \quad y(t) = A_y \sin(\omega_y t + \phi_y),$$

where A_x and A_y are the amplitudes, ω_x and ω_y are the angular frequencies, and ϕ_x and ϕ_y are the phases. For the curve at hand, the geometric primitives are

$$x(t) = \sin\left(3t + \frac{\pi}{2}\right), \quad y(t) = \sin(4t), \quad z(t) = 0. \tag{1}$$

The generation of the path points is done by means of the `path()` function, which is iterated for 100 points. In order to have the trajectory start and end in the center of the figure, the waypoints space from $t = \frac{\pi}{2}$ to $t = \frac{5\pi}{2}$. To better fit the workspace, the trajectory has been scaled by multiplying the coordinates by a set `scale_factor` equal to 0.15 and by translating the origin to `start_pose[:3]`.

2.2 Trajectory generation

The final trajectory has been made by interpolating the path with 300 points for an execution time of 60 seconds. A cubic spline interpolation has been done using the `CubicSpline()` method with `clamped` boundary conditions to ensure that the velocity is null at the extremes of the trajectory. A fixed orientation `start_pose[3:]` has been set for all the interpolated points.

A plot of the waypoints and the resulting interpolation is shown in Figure 2.

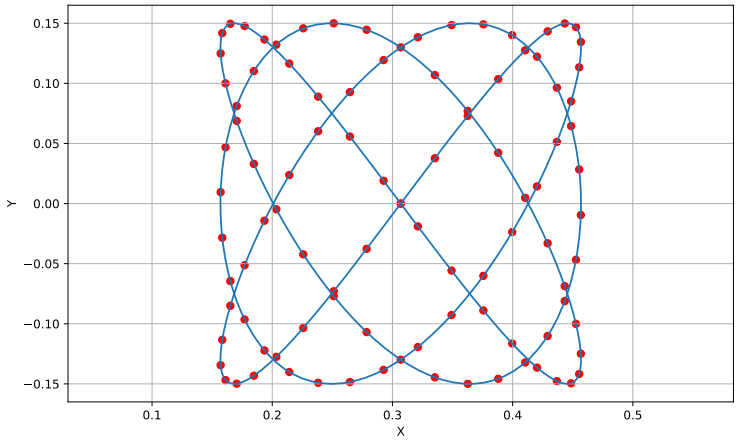


Figure 2: top view of waypoints (red dots) and interpolated trajectory (blue line).

3 CONTROLLER SELECTION AND IMPLEMENTATION

Since the trajectory has been planned in the task space, the *Cartesian impedance example controller* from the package `franka_example_controllers` has been chosen. It was implemented by specifying the `controller` argument in `panda_control.launch`.

The controller moves the end-effector to a desired pose by applying suitable torques[1] . Let

$$\tilde{\mathbf{p}} = \mathbf{p} - \mathbf{p}_d \tag{2}$$

be the *position error* between the actual position $\mathbf{p}$ and the desired one $\mathbf{p}_d$, and let

$$\tilde{\mathbf{q}} = \mathrm{diff}(\mathbf{q}, \mathbf{q}_d) \tag{3}$$

be the *difference quaternion* between the actual orientation $\mathbf{q}$ and the desired one $\mathbf{q}_d$. The controller applies a torque

$$\boldsymbol{\tau}_{\text{task}} = \mathbf{J}^\top \left(-\mathbf{K}_x \begin{bmatrix} \tilde{\mathbf{p}} \\ \tilde{\mathbf{q}} \end{bmatrix} - \mathbf{D}_x \mathbf{J} \dot{q} \right), \tag{4}$$

where $\mathbf{J}$ is the Jacobian matrix, $\mathbf{K}_x$ is the Cartesian stiffness matrix, $\mathbf{D}_x$ is the Cartesian damping matrix, and $\dot{q}$ is the joints velocity. To avoid unwanted movements, a PD control on the kernel of the Jacobian is applied:

$$\boldsymbol{\tau}_{\text{null}} = (\mathbf{I} - \mathbf{J}^\top \mathbf{J}^\dagger)(\mathbf{K}_q(q_d - q) - \mathbf{D}_q \dot{q}), \tag{5}$$

where $\mathbf{K}_q$ and $\mathbf{D}_q$ are, respectively, the stiffness matrix and the damping matrix of the kernel, $\mathbf{J}^\dagger$ is the pseudo-inverse of the Jacobian, and $q_d - q$ is the difference between the desired configuration q_d and the real one q. The resulting torque is

$$\boldsymbol{\tau}_d = \boldsymbol{\tau}_{\text{task}} + \boldsymbol{\tau}_{\text{null}} + \boldsymbol{\tau}_{\text{Coriolis}}, \tag{6}$$

where $\boldsymbol{\tau}_{\text{Coriolis}}$ is got from the robot dynamics using `getCoriolis()`. To have smooth torque transitions, a saturation algorithm is applied to $\boldsymbol{\tau}_d$ in order to minimize the variation w.r.t. the desired joint torques from the previous control cycle (`tau_J_d`).

4 EXECUTION AND DATA COLLECTION

This step has required the creation of the `trajectory_publisher.py` and `data_collector.py`; the first node sends the desired poses to the robot, while the second one collects data on end-effector poses and joint positions and velocities.

The simulation has been started by launching `panda_control.launch`, which loads the Franka Emika Panda robot in Gazebo with the chosen controller; then, the two nodes have been started.

4.1 Trajectory execution

The node `trajectory_publisher` loads the poses in `trajectory.csv` and publishes them to the topic `/cartesian_impedance_example_controller/equilibrium_pose` according to the provided timing. Also, the node commands `data_collector` to start and stop data collection by publishing specific messages to `/data_collection`. Before starting the execution, the node `/interactive_marker` had to be killed to avoid conflicts, since it sends pose messages to the same topic of the equilibrium pose.

[1] `docs.ros.org/en/noetic/api/franka_example_controllers/html/cartesian__impedance__example_controller_8cpp_source.html`

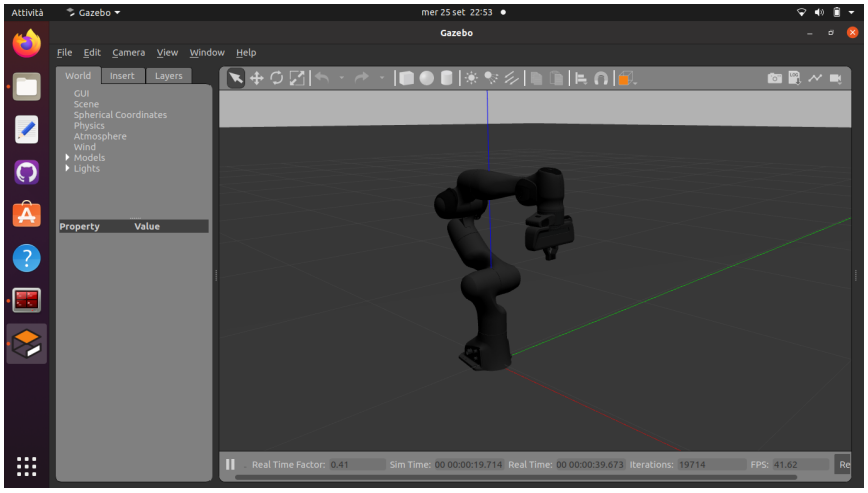


Figure 3: Gazebo's GUI with the Franka Emika Panda robot loaded.

4.2 Data collection

The node `data_collector`, upon receiving a `start` message from `/data_collection`, collects data on joint positions and velocities by subscribing to `/joint_states` and creates a listener to get the homogeneous transformation (rotation and translation) between the base link and the end-effector. At the end of the execution, a `stop` message is received and data is stored in `joint_positions.csv`, `joint_velocities.csv`, and `end_effector_pose.csv`.

This node has to be executed right after `trajectory_publisher`.

5 DATA ANALYSIS

By comparing the recorded positions of the end-effector with the one from the computed trajectory (Figure 4), one can see that the error between the desired position and the "true" one grows as the end-effector goes farther from the origin of the reference frame. Although the error is in the order of the centimeter, the figure is still easily recognizable and, as we can see from Figure 5, there is quasi-null variation of the z coordinates.

There may be several reasons behind this such as, just to name a few, growing difficulties in compensating errors and the action of gravity as far the end-effector goes, and inaccuracies of the model for points of the task space near to the border of the workspace.

The orientation (Figure 6) is shown to have oscillatory behavior on all the four components q_x, q_y, q_z, q_w, although they were supposed to be constant. However, they are negligible, since the amplitudes are quite low.

6 CONCLUSION

Planning a trajectory directly in the task space has required the implementation of a Cartesian impedance controller, whose use results in a smooth trajectory execution with negligible oscillations on geometric variables that have been set to be constant. The main challenge was just the choice of the controller: using a PID controller on the joints positions would have been a simpler approach for controlling the robot, since one could directly command their angles, but it would have required the implementation of an inverse kinematics algorithm on

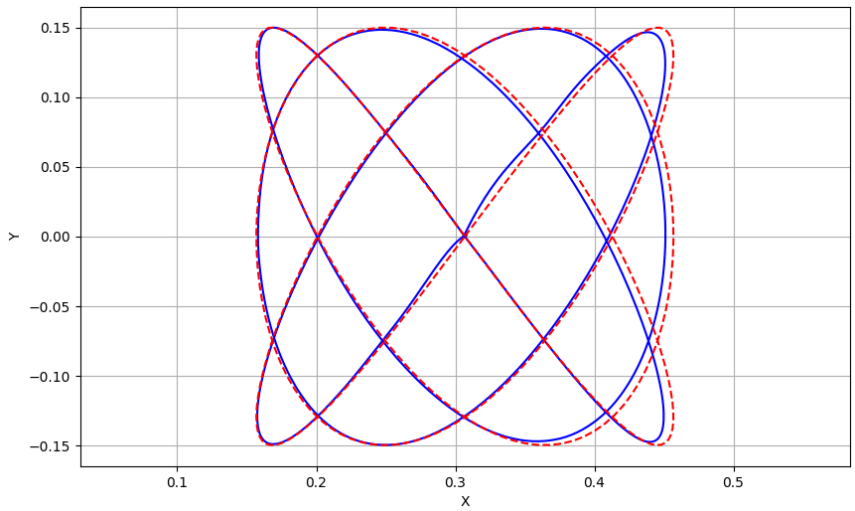


Figure 4: 2D comparison of the recorded positions (blue) with the computed ones (red).

the Cartesian poses, with related issues of numerical instability and low accuracy. Hence, a controller that directly acts on the end-effector has been chosen, and it proved to follow the trajectory in an appreciable way, although it gets less precise as the distance of the end-effector from the origin of the reference frame grows. This can be overcome by suitably tuning the stiffness and damping matrices, paying attention for the execution to not result in "jerkier" movements, or by designing a smaller trajectory with a center closer to the origin of the reference frame.

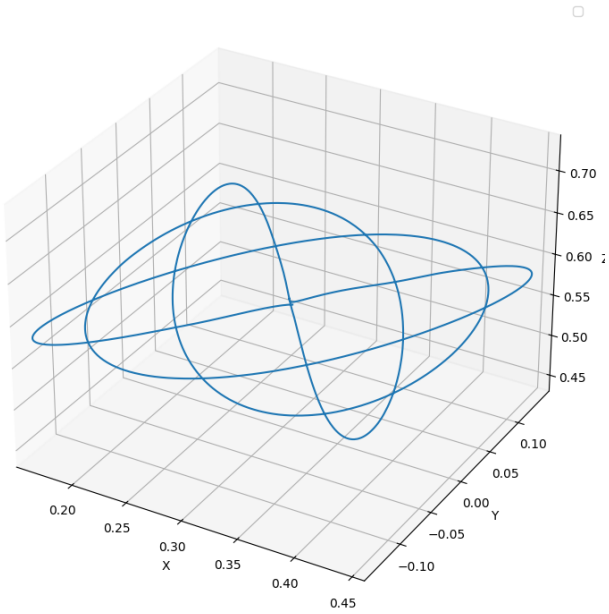


Figure 5: 3D plot of the recorded positions.

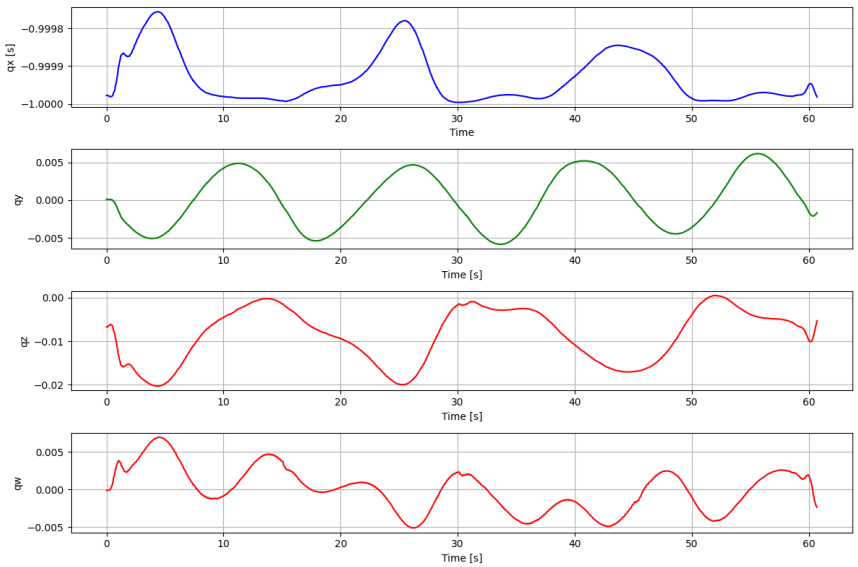


Figure 6: components of q_x, q_y, q_z, and q_w over time.